

AdaptivePager: Learning-Augmented Page Replacement Under Workload Shift

Nitun Kundu Swapnil

Roll: 202414036

Section: A

CSE-307: Operating Systems

GitHub Repository: github.com/nkswapnil087/AdaptivePager

Submission Package: Part I - Term Paper | Part II - Walkthrough Notes

Contents

Part I: Term Paper	2
Abstract	2
1 Problem Framing and Motivation	2
2 Algorithms and Learned Component	2
3 Implementation Structure	3
4 Experimental Design	3
5 Results and Discussion	4
6 Limitations and Future Work	5
7 Conclusion	6
Part II: Implementation Walkthrough	7

Part I: Term Paper

Abstract

Page replacement is a central memory-management problem: when all physical frames are occupied and a referenced page is absent, the operating system must choose a resident page to evict. Classical policies such as FIFO and LRU use fixed heuristics, while Belady’s Optimal policy chooses the page whose next reference is furthest in the future and therefore provides an ideal offline benchmark. This project, *AdaptivePager*, investigates whether a lightweight decision-tree model can learn useful eviction behavior from observable history and remain effective after the access pattern changes. The model is trained from Belady-labelled eviction examples but receives only two past-observable features: recency and recent access frequency. Experiments use reproducible 400-reference synthetic traces with a deliberate midpoint shift from a small locality-heavy working set to wider bursty/random behavior. Across seven unseen test traces, Optimal averaged 97.71 page faults, compared with 157.86 for LRU, 160.14 for FIFO, and 175.71 for the learned policy. The result shows that learning from limited historical signals does not automatically outperform a strong classical heuristic. The most important explanation is an information asymmetry: Belady generates labels using perfect future knowledge, whereas the learned policy must infer decisions from the past alone.

1 Problem Framing and Motivation

Virtual memory allows a process to operate with an address space larger than the physical memory currently available. When a referenced page is not resident, a page fault occurs; if no free frame exists, a replacement policy must select a victim page. The quality of this decision affects the number of future faults and therefore memory-system performance. Belady’s classic work formalized replacement algorithms for virtual-storage systems and established the value of future-aware replacement as a theoretical benchmark [1]. In real execution, however, future references are unknown.

Practical replacement therefore depends on observable behavior. LRU is motivated by locality: recently used pages are often likely to be useful again. Denning’s working-set model similarly characterizes a process through recently referenced pages and uses recent behavior as an approximation of current memory demand [2]. Yet locality is not permanent. Programs move between phases, data structures change, and an access stream that is predictable for one interval can become wider or more bursty later.

AdaptivePager studies this gap through a small learning-augmented experiment. The research question is: *Can a lightweight learned eviction heuristic, trained from Optimal decisions but restricted to past-observable information, make competitive replacement decisions when the workload shifts?* A second objective is to understand why the learned method succeeds or fails rather than assuming that machine learning must outperform classical algorithms.

2 Algorithms and Learned Component

Four policies are evaluated with the same number of frames and the same reference traces.

FIFO evicts the page that has been resident for the longest time. Its decision depends only on arrival order and does not consider how often or how recently a page is used.

LRU evicts the resident page whose most recent reference is oldest. It directly exploits temporal locality and therefore represents a stronger history-based classical baseline.

Optimal (Belady) looks forward in the complete trace and evicts the resident page whose next use is furthest in the future; a page that is never referenced again is preferred immediately. Because a real operating system cannot know future requests, this policy is used as an offline ideal rather than a deployable mechanism [1].

Learned policy uses a `DecisionTreeClassifier` from scikit-learn [3]. At each full-memory page fault during training, Belady identifies the ideal victim. Every current resident page becomes one training example with two features: (1) *recency*, the number of references since that page was last accessed, and (2) *recent frequency*, the number of occurrences in the previous 20 references. The Belady-selected victim receives label 1; the remaining candidates receive label 0. The tree is deliberately small (maximum depth 5 and minimum leaf size 8) so that the learned behavior remains lightweight and explainable.

During evaluation, Belady is removed completely from the learned policy. The tree scores every resident candidate using only the already observed prefix of the trace. The candidate with the highest predicted probability of being a

good victim is evicted. This separation is crucial: future references are used only to produce training labels, never as model input.

3 Implementation Structure

The project is organized into small Python modules so that each experimental responsibility is visible and independently testable.

policies.py. This module implements FIFO, LRU, and Optimal. It also defines a per-access event record containing the current page, hit/fault status, evicted page, and resulting frame contents. These event records allow the experiment to reconstruct metrics for any portion of a trace rather than only reporting a final total. The same module contains `belady_victim()`, which scans future references to choose the Optimal victim and is reused only when generating supervised training labels.

features.py. This module defines the learned policy’s observable information. For each resident candidate it computes recency and frequency over a fixed recent-history window. An explicit validation rule requires the supplied past trace to end immediately before the current index, making future-information leakage difficult to introduce accidentally. In effect, this file defines what the learned model is allowed to “see.”

learned.py. This module builds Belady-labelled candidate examples, trains the Decision Tree, and runs learned inference. Training proceeds through multiple independent synthetic traces. Whenever memory is full and a fault occurs, Belady chooses the ideal victim, but feature extraction receives only the past prefix. During inference, the model evaluates all current residents and selects the highest-scoring candidate. A deterministic tie rule keeps runs reproducible.

workload.py. This module generates the controlled workload shift. The default trace contains 400 references and changes behavior at index 200. Phase A mostly cycles through a four-page working set, includes additional repeats of two hot pages, and occasionally accesses nearby noise pages. Phase B expands to sixteen possible pages, selects temporary five-page burst sets for 10–24 accesses, and mixes these bursts with wider random references. The generator accepts a random seed so that each experiment can be reproduced exactly.

experiment.py. This module coordinates the full pipeline. It creates independent training traces, fits the learned pager once, generates unseen test traces, runs all four policies on each identical trace, and computes metrics separately before the shift, after the shift, and overall. Detailed and aggregate CSV files are produced, along with comparison figures. The default evaluation uses four frames, ten training traces, and seven test seeds.

4 Experimental Design

Training and testing are intentionally separated. Ten training traces use seeds 10042–10051, while evaluation uses seeds 42–48. Thus the model never evaluates on the exact sequences from which its labels were learned. All policies receive the same trace for a given seed, the same four-frame capacity, and the same shift location. This controlled setup isolates the replacement policy as the primary changing factor.

The principal metrics are page-fault count and hit ratio. Because each reference must be either a hit or a fault, the implementation also verifies that hits + faults = accesses. Metrics are reported for three scopes: the first 200 accesses (before shift), the final 200 accesses (after shift), and the complete 400-reference trace. Automated tests verify the classical algorithms on small known cases; the final implementation passed 12 tests before the reported experiment was generated.

Table 1: Primary demonstration run (seed 42, four frames).

Policy	Before faults	After faults	Overall faults	Hit ratio
FIFO	34	115	149	0.6275
LRU	32	115	147	0.6325
Optimal	17	77	94	0.7650
Learned	35	124	159	0.6025

5 Results and Discussion

Table 1 shows a clear shift effect. With seed 42, FIFO faults rise from 34 before the shift to 115 after it; LRU rises from 32 to 115; Optimal rises from 17 to 77; and the learned policy rises from 35 to 124. The second half is deliberately harder: a four-frame memory can no longer hold the larger, changing burst sets, and recent history becomes less predictive. Figure 1 makes this degradation visible.

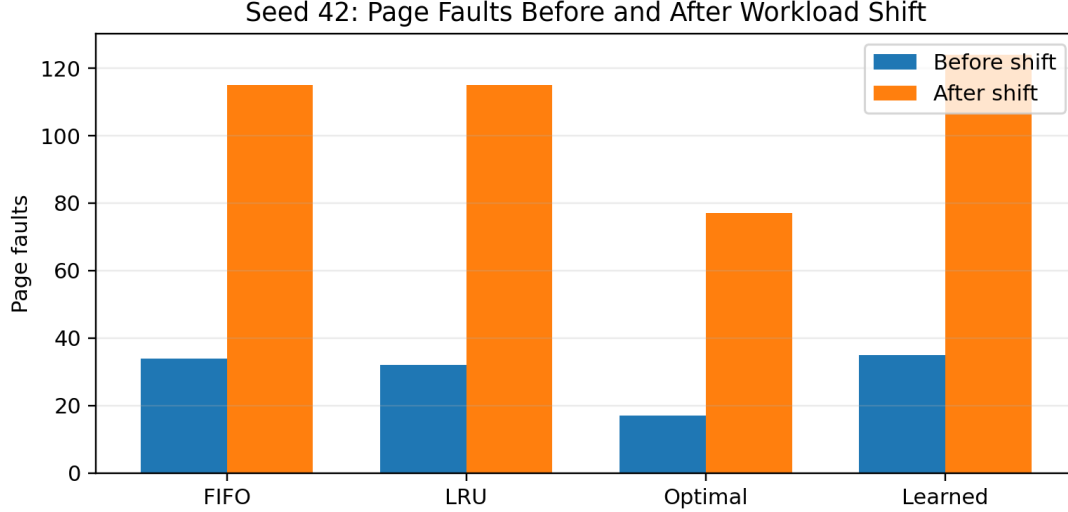


Figure 1: Page faults before and after the workload shift for the primary trace.

The seven-run aggregate result reinforces the same ordering. Mean overall faults were 97.71 for Optimal, 157.86 for LRU, 160.14 for FIFO, and 175.71 for the learned policy (Figure 2). The learned method therefore did not outperform the classical heuristics in this experiment. This outcome is not treated as a failed experiment; it exposes several important limitations of the learning setup.

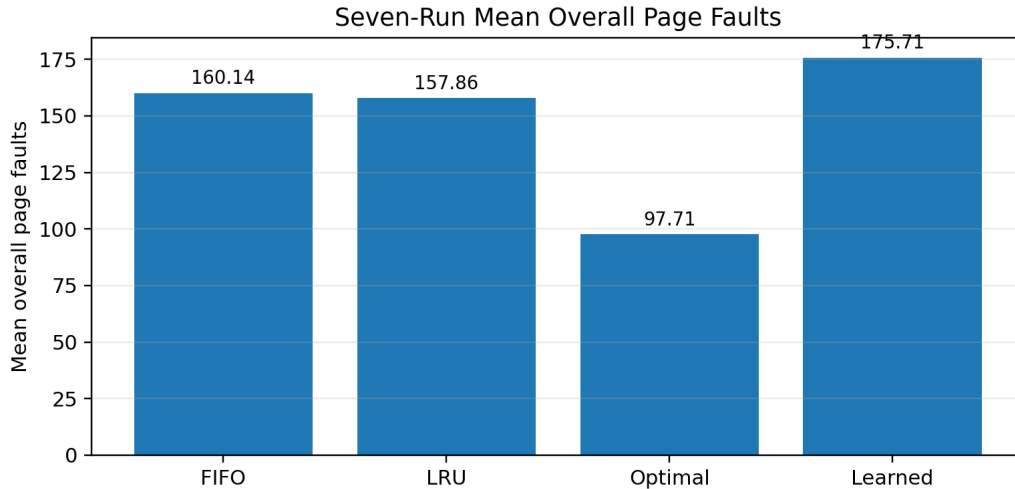


Figure 2: Mean overall page faults across seven unseen evaluation seeds. Lower is better.

1) Future-information asymmetry is the central result. Belady generates the target label by inspecting the future, while the learned model receives only historical recency and frequency. Therefore the model is asked to imitate decisions whose true cause is partly hidden from it. Two resident pages can look nearly identical from the past while having very different future next-use distances. Belady distinguishes them perfectly; the Decision Tree cannot. In short, the teacher provides the correct answer but the student is not given all the information the teacher used to

obtain it. This is the strongest explanation for the large Optimal–Learned gap.

2) The feature set is intentionally narrow. Recency and recent frequency are meaningful historical signals, but they are only proxies for future usefulness. The model may learn that high-recency, low-frequency pages are often Belady victims, yet this is not Belady’s actual rule. Belady’s rule is directly future-based. A richer historical representation could include multiple frequency windows, inter-reference gaps, or trend information, but adding features would also increase model complexity and move away from the lightweight objective of the assignment.

3) Workload shift weakens learned associations. Phase A repeatedly revisits a small working set, so historical statistics are comparatively informative. Phase B expands the active page range and replaces stable locality with short-lived bursts and wider random references. Relationships learned under more regular behavior therefore become less reliable. This is a small example of distribution shift: a pattern that worked in one region of the trace does not necessarily remain predictive after the workload changes.

4) Training-state mismatch can compound mistakes. Training states are produced while Belady itself performs the evictions. Consequently, the model learns from frame configurations created by near-perfect decisions. During testing, the learned model controls its own frame state. One poor eviction changes which pages remain resident, which changes the next decision context, potentially creating states unlike those observed during Belady-generated training. Errors can therefore have downstream effects rather than remaining isolated.

5) LRU is a strong baseline under locality. The learned model is not competing against an arbitrary rule; it is competing against a heuristic built around one of the most useful properties of real program behavior. The working-set concept specifically exploits recent references as evidence of current memory demand [2]. Therefore a small Decision Tree with only two features should not be expected to beat LRU automatically.

These findings support a cautious conclusion: learning-augmented replacement can be studied meaningfully with lightweight supervised models, but prediction quality depends on both feature information and workload stability. “Using machine learning” is not itself an advantage; the model must receive signals that are sufficient for the decision it is being asked to approximate.

6 Limitations and Future Work

The experiment uses synthetic traces rather than traces collected from production applications, and the learned model uses only a shallow decision tree with two historical features. The current classifier also predicts whether each page resembles Belady’s victim independently, rather than directly optimizing long-horizon page-fault count. These choices keep the experiment understandable but limit predictive power.

Proposed Future Work - Controlled Lookahead Experiment. The clearest next step is to give the learned policy a small, controlled amount of future information and measure how its performance changes. The experiment would compare **Learned-0** (no future references, the current model), **Learned-1**, **Learned-5**, and **Learned-10** (access to the next 1, 5, or 10 references), with **Belady** representing complete future knowledge. For each level, page faults, hit ratio, and the remaining performance gap from Optimal would be measured. This would directly test the central conclusion of AdaptivePager: **how much of Belady’s advantage comes from access to future information itself?**

This extension would be exploratory rather than a realistic deployable OS policy, because real online operating systems normally do not know future references. Its purpose is to isolate and quantify the value of future information rather than to propose lookahead as a practical replacement mechanism.

7 Conclusion

AdaptivePager implemented FIFO, LRU, Optimal, and a Decision Tree-based page-replacement policy and evaluated them under a deliberate workload shift. All methods degraded after the shift, showing the cost of a larger and less stable working pattern. Optimal remained substantially better because it possesses information unavailable to real online policies. The learned method, despite being trained from Optimal labels, performed worse than LRU and FIFO across the seven test traces. The crucial conclusion is therefore not that the model “failed,” but that imitation is constrained by information: **the learned policy reasons from the past, while Belady already knows the future**. The experiment demonstrates why classical heuristics remain competitive and why a learned system must be judged not only by its model type, but also by the quality and relevance of the information it can observe.

References

- [1] L. A. Belady, “A Study of Replacement Algorithms for a Virtual-Storage Computer,” *IBM Systems Journal*, vol. 5, no. 2, pp. 78–101, 1966. doi:10.1147/SJ.52.0078.
- [2] P. J. Denning, “The Working Set Model for Program Behavior,” *Communications of the ACM*, vol. 11, no. 5, pp. 323–333, 1968. doi:10.1145/363095.363141.
- [3] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

Part II: Implementation Walkthrough

The screenshots below show the project from source structure to execution, results, and final verification.

1. Project Structure

The project is split into separate modules for workload generation, classical policies, learned replacement, features, plotting, tests, and saved results.

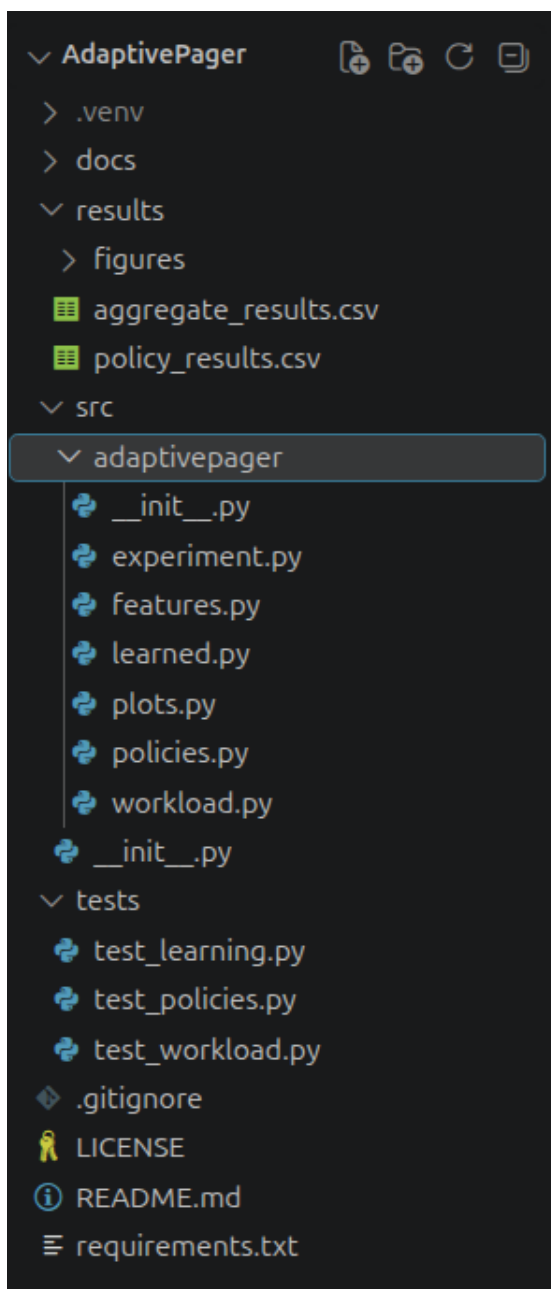


Figure 3: AdaptivePager project structure in VS Code.

2. Generating the Workload Shift

The 400-reference trace changes at index 200: a small locality-heavy working set is followed by a wider bursty/random phase.

```
1  from dataclasses import dataclass
2
3  import numpy as np
4
5
6  @dataclass(frozen=True)
7  class ShiftWorkload:
8      trace: tuple[int, ...]
9      shift_index: int
10     seed: int
11
12
13 def generate_shift_workload(length: int = 400, seed: int = 42) -> ShiftWorkload:
14     """Generate a locality-heavy phase followed by a wider, bursty phase.
15
16     Phase A mostly advances through a four-page working set, with hot-page
17     repeats and occasional nearby noise. Phase B switches among short-lived
18     five-page burst sets selected from sixteen pages, plus global random reads.
19     """
20     if length < 20:
21         raise ValueError("length must be at least 20")
22     rng = np.random.default_rng(seed)
23     shift_index = length // 2
24
25     phase_a: list[int] = []
26     local_pages = np.array([0, 1, 2, 3])
27     cursor = int(rng.integers(0, len(local_pages)))
28     for _ in range(shift_index):
29         choice = rng.random()
30         if choice < 0.72:
31             page = int(local_pages[cursor])
32             cursor = (cursor + 1) % len(local_pages)
33         elif choice < 0.94:
34             page = int(rng.choice(local_pages[:2]))
35         else:
36             page = int(rng.choice([4, 5]))
37         phase_a.append(page)
38
39     phase_b: list[int] = []
40     wide_pages = np.arange(16)
41     burst_pages = rng.choice(wide_pages, size=5, replace=False)
42     burst_remaining = 0
43     for _ in range(length - shift_index):
44         if burst_remaining == 0:
45             burst_pages = rng.choice(wide_pages, size=5, replace=False)
46             burst_remaining = int(rng.integers(10, 25))
47         if rng.random() < 0.68:
48             page = int(rng.choice(burst_pages))
49         else:
50             page = int(rng.choice(wide_pages))
51         phase_b.append(page)
52         burst_remaining -= 1
53
54     return ShiftWorkload(tuple(phase_a + phase_b), shift_index, seed)
55
```

Figure 4: workload.py: Phase A and Phase B generation in the actual source file.

3. Training the Learned Policy

Belady supplies the ideal victim label, while the Decision Tree receives only features calculated from the observed prefix of the trace.

```
for index, page in enumerate(trace):
    if page not in frames:
        if len(frames) == frame_count:
            # This is the sole future-aware operation in ML training.
            ideal_victim = belady_victim(frames, trace, index)
            observed_prefix = trace[:index]
            for candidate in frames:
                feature_rows.append(
                    candidate_features(
                        candidate,
                        index,
                        last_seen,
                        observed_prefix,
                        window_size,
                    )
                )
            labels.append(int(candidate == ideal_victim))
            frames.remove(ideal_victim)
        frames.append(page)
    last_seen[page] = index
```

Figure 5: Belady-labelled training examples built from past-only features.

4. Learned Eviction During Testing

During testing, Belady is absent. The model scores each resident page with `predict_proba()` and evicts the highest-scoring candidate.

```
for index, page in enumerate(trace_list):
    hit = page in frames
    evicted = None
    if not hit:
        if len(frames) == self.frame_count:
            observed_prefix = trace_list[:index]
            feature_rows = [
                candidate_features(
                    candidate,
                    index,
                    last_seen,
                    observed_prefix,
                    self.window_size,
                )
                for candidate in frames
            ]
            positive_column = list(self.classifier.classes_).index(1)
            scores = self.classifier.predict_proba(feature_rows)[
                :, positive_column
            ]
            # Ties prefer greater recency, then the earlier frame.
            victim_index = max(
                range(len(frames)),
                key=lambda position: (
                    scores[position], feature_rows[position][0], -position
                ),
            )
            evicted = frames.pop(victim_index)
        frames.append(page)
        last_seen[page] = index
        events.append(AccessEvent(index, page, hit, evicted, tuple(frames)))
```

Figure 6: Past-only learned victim selection during inference.

5. Running the Experiment

The experiment trains on 3,872 candidate examples, evaluates unseen seeds 42–48, and writes the result files and figures automatically.

```
● (.venv) nk-swapnil@nk-swapnil-L0Q-15IAX9E:~/Desktop/AdaptivePager$ python3 -m src.adaptivepager.experiment
Trained DecisionTreeClassifier on 3,872 candidate examples from seeds 10042-10051.
Evaluated seeds [42, 43, 44, 45, 46, 47, 48]; primary demonstration seed: 42.

Primary trace results:
policy      phase  hits  faults  hit_ratio
FIFO before_shift  166    34    0.830
FIFO after_shift   85   115    0.425
FIFO overall      251   149    0.627
LRU before_shift  168    32    0.840
LRU after_shift   85   115    0.425
LRU overall      253   147    0.632
Optimal before_shift 183    17    0.915
Optimal after_shift 123    77    0.615
Optimal overall     306    94    0.765
Learned before_shift 165    35    0.825
Learned after_shift  76   124    0.380
Learned overall     241   159    0.603

Wrote results to /home/nk-swapnil/Desktop/AdaptivePager/results
Figures: page_faults_by_phase.png, hit_ratio_by_phase.png
○ (.venv) nk-swapnil@nk-swapnil-L0Q-15IAX9E:~/Desktop/AdaptivePager$
```

Figure 7: Actual terminal run of `python3 -m src.adaptivepager.experiment`.

6. Page-Fault Result

Every policy produces more faults after the shift. Optimal stays lowest, while the learned policy records the highest post-shift fault count in the primary run.

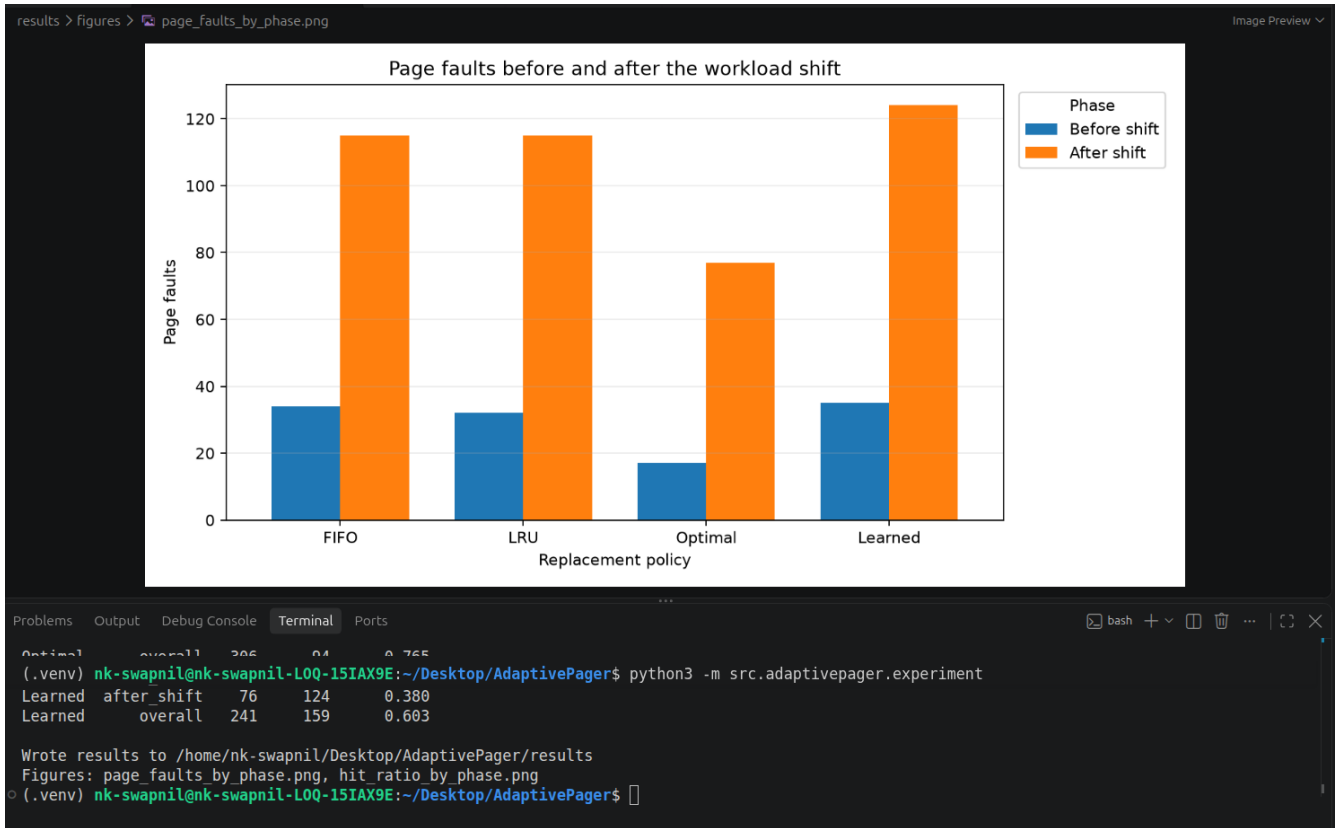


Figure 8: Generated page-fault comparison displayed inside the same VS Code session.

7. Hit-Ratio Result and Verification

Hit ratios fall after the workload shift, matching the page-fault result. The final automated test run also confirms that all 12 project tests pass.

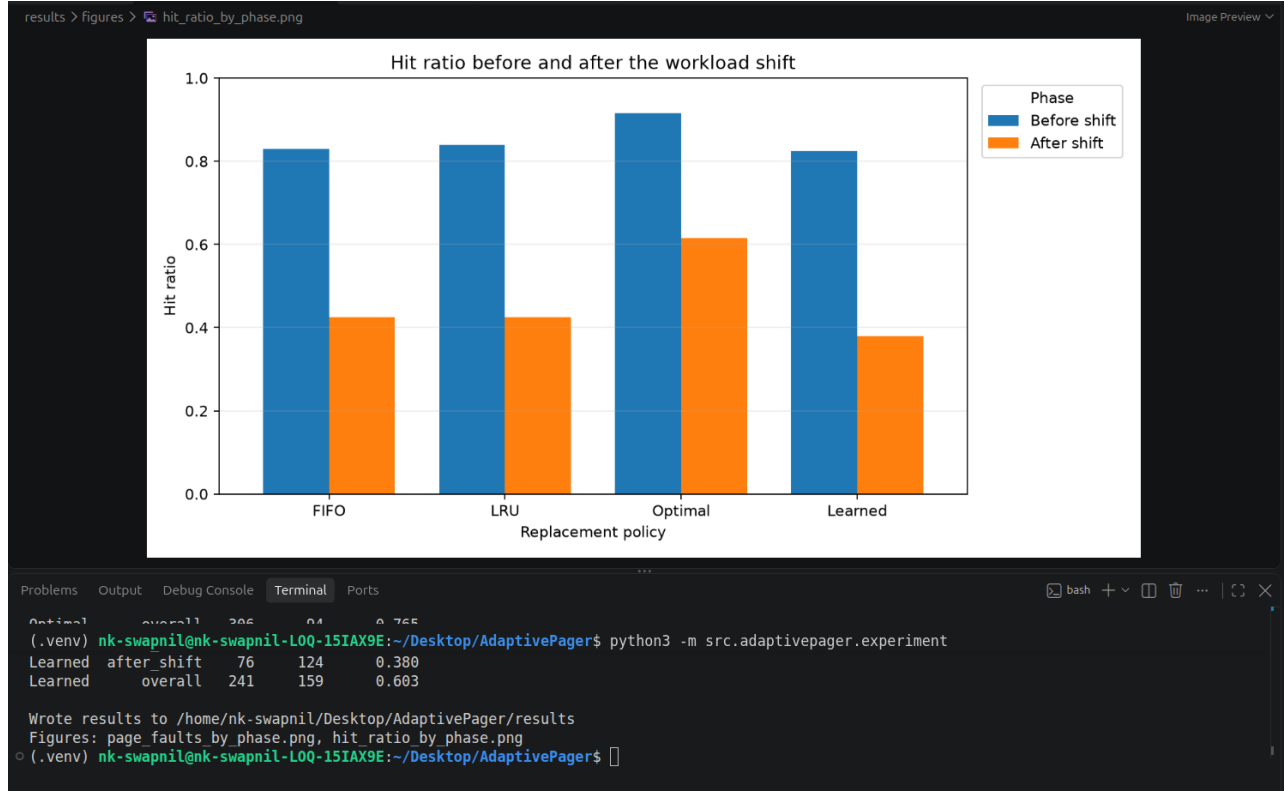


Figure 9: Generated hit-ratio comparison.

```
Problems Output Debug Console Terminal Ports
(.venv) nk-swapnil@nk-swapnil-L0Q-15IAX9E:~/Desktop/AdaptivePager$ python3 -m pytest -q
[100%]
12 passed in 0.63s
(.venv) nk-swapnil@nk-swapnil-L0Q-15IAX9E:~/Desktop/AdaptivePager$
```

Figure 10: Final verification: 12 tests passed.

Walkthrough takeaway: the screenshots show the full path from workload generation and Belady-labelled training to past-only learned inference, actual execution, generated results, and automated verification.